Code Explanation

Data Aggregation Module Explanation

The provided code is a data aggregation module for an ETL (Extract, Transform, Load) pipeline, written in Python using the PySpark library. It contains four main functions that aggregate sales data from a given DataFrame and return new DataFrames with various metrics.

Overview of the Code
The code is designed to process sales data and generate aggregated metrics for sales summary, product analytics, customer analytics, and time series analytics. Each function takes an enriched sales DataFrame as input and returns a new DataFrame with the corresponding aggregated metrics.

Step 1: Importing Necessary Libraries and Defining the create_sales_summary Function
This step involves importing the necessary libraries, including PySpark's DataFrame and functions modules. The `create_sales_summary` function is defined to create a summary of sales data.

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F

def create_sales_summary(sales_df: DataFrame) -> DataFrame:
    """
    Create a summary of sales data.

    Args:
        sales_df: Enriched sales DataFrame

    Returns:
        Summary DataFrame with aggregated metrics
    """
```

Step 2: Aggregating Sales Data for Sales Summary
In this step, the `create_sales_summary` function groups the sales data by "date", "region", and "category" columns and applies various aggregation functions to calculate metrics such as total quantity, total sales, transaction count, unique customers, and average price.

```
summary_df = sales_df.groupBy("date", "region", "category").agg(
    F.sum("quantity").alias("total_quantity"),
    F.sum("total_amount").alias("total_sales"),
    F.count("sale_id").alias("transaction_count"),
    F.countDistinct("customer_id").alias("unique_customers"),
    F.avg("unit_price").alias("average_price")
)
return summary_df
```

Step 3: Defining the create_product_analytics Function and Aggregating Product Data
The `create_product_analytics` function is defined to create product analytics from sales data. It groups the sales data by "product_id", "product_name", and "category" columns and calculates metrics such as total quantity sold, total revenue, average price, unique customers, and days sold.

```
def create_product_analytics(sales_df: DataFrame) -> DataFrame:
    product_analytics = sales_df.groupBy("product_id", "product_name
", "category").agg(
        F.sum("quantity").alias("total_quantity_sold"),
        F.sum("total_amount").alias("total_revenue"),
        F.avg("unit_price").alias("average_price"),
        F.countDistinct("customer_id").alias("unique_customers"),
        F.countDistinct("date").alias("days_sold")
    ).withColumn(
        "revenue_per_day", F.col("total_revenue") / F.col("days_sold
")
    )
    return product_analytics
```

Step 4: Defining the create_customer_analytics Function and Aggregating Customer Data
The `create_customer_analytics` function is defined to create customer analytics from sales data. It groups the sales data by "customer_id", "customer_name", and "region" columns and calculates metrics such as total spend, transaction count, shopping days, unique products, and last purchase date.

```
def create_customer_analytics(sales_df: DataFrame) -> DataFrame:
    customer_analytics = sales_df.groupBy("customer_id", "customer_n
ame", "region").agg(
        F.sum("total_amount").alias("total_spend"),
        F.count("sale_id").alias("transaction_count"),
        F.countDistinct("date").alias("shopping_days"),
        F.countDistinct("product_id").alias("unique_products"),
        F.max("date").alias("last_purchase_date")
    ).withColumn(
        "average_transaction_value", F.col("total_spend") / F.col("t
ransaction_count")
    )
    return customer_analytics
```

Step 5: Defining the create_time_series_analytics Function and Aggregating Time Series Data
The `create_time_series_analytics` function is defined to create time series analytics from sales data. It groups the sales data by "date" and "region" columns and calculates metrics such as daily sales, transaction count, unique customers, and average basket size.

```
def create_time_series_analytics(sales_df: DataFrame) -> DataFrame:
    time_series = sales_df.groupBy("date", "region").agg(
        F.sum("total_amount").alias("daily_sales"),
        F.count("sale_id").alias("transaction_count"),
        F.countDistinct("customer_id").alias("unique_customers"),
        F.avg("total_amount").alias("average_basket_size")
    )
    return time_series
```